**UNIVERSIDAD TÉCNICA DE AMBATO**

Algoritmos y Lógica de Programación

Informe de exposición grupal

# Lectura de archivos y procesamiento de información

## en Java

**Grupo 5**

Sandra Alulema
Anthony Armas
Joselyn Guaman
Valentina Ortiz
Andrés Vayas

Ambato, Ecuador · Noviembre de 2026

# Contenido

**1.** Objetivos

**2.** Desarrollo

Tema 1. Fundamentos y lectura de archivos

Tema 2. Escritura y procesamiento de información

**3.** Conclusiones

**4.** Recomendaciones

# 1. Objetivos

## Objetivo general

Comprender y aplicar los conceptos y las herramientas del paquete `java.io` para leer, escribir y procesar información almacenada en archivos, de modo que los programas desarrollados en Java puedan conservar y reutilizar datos más allá de su ejecución.

## Objetivos específicos

- Explicar qué son los archivos, sus tipos y la diferencia entre la memoria RAM y el almacenamiento permanente, identificando cómo Java accede a ellos mediante rutas, la clase `File` y los lectores `FileReader` y `BufferedReader`.
- Aplicar las clases `FileWriter` y `BufferedWriter` para escribir información en archivos, distinguiendo entre sobrescribir y agregar contenido y gestionando las excepciones asociadas.
- Desarrollar programas que sigan el flujo entrada, procesamiento y salida con datos de archivos, incorporando validación, conversión de texto a números, búsqueda y filtrado de información.

# 2. Desarrollo

## Tema 1. Fundamentos y lectura de archivos

### 1.1 Qué son los archivos, para qué sirven y tipos de archivos

Un archivo es un conjunto de datos relacionados que se almacena en un dispositivo con un nombre y una ubicación determinados. Sirve para guardar información de forma duradera: documentos, configuraciones, registros de ventas, imágenes o los propios programas. Gracias a ellos, la información no desaparece cuando se cierra una aplicación.

Según su contenido, los archivos se agrupan en cuatro tipos principales:

- **Texto:** contienen caracteres legibles (.txt, .csv, .java). Pueden abrirse con cualquier editor.
- **Imágenes:** almacenan datos gráficos (.png, .jpg, .gif).
- **Documentos:** combinan texto, formato y otros elementos (.pdf, .docx, .xlsx).
- **Binarios:** guardan los datos tal como los entiende la máquina (.exe, .class, .jar) y no son legibles directamente.

### 1.2 Diferencia entre RAM y almacenamiento permanente, y cómo funciona la lectura de archivos

La memoria RAM es rápida pero volátil: guarda las variables y los objetos mientras el programa se ejecuta y pierde todo al cerrarlo o apagar el equipo. El almacenamiento permanente (disco duro, SSD, memoria USB) es más lento, pero conserva los datos aunque no haya energía.

Leer un archivo significa copiar su contenido desde el almacenamiento permanente hacia la RAM para que el programa pueda trabajar con él. El proceso consta de tres pasos: abrir el archivo, recorrer su contenido y cerrarlo para liberar el recurso.

| Característica | RAM | Almacenamiento permanente |
|---|---|---|
| Persistencia | Volátil | Se conserva sin energía |
| Velocidad | Muy alta | Menor |
| Uso en Java | Variables y objetos | Archivos (.txt, .csv, etc.) |

### 1.3 Leer, escribir y almacenar información; métodos read() y readLine()

Leer es traer datos del archivo al programa; escribir es enviar datos del programa al archivo; almacenar es dejarlos guardados de manera permanente. Para la lectura, Java ofrece dos métodos fundamentales:

- `read()` lee **un carácter** a la vez y lo devuelve como un entero; al llegar al final del archivo devuelve `-1`.
- `readLine()` lee **una línea completa** como `String`; al llegar al final devuelve `null`.

```
// read(): carácter por carácter
FileReader fr = new FileReader("saludo.txt");
int c;
while ((c = fr.read()) != -1) {
    System.out.print((char) c);
}
fr.close();
```

### 1.4 Paquete java.io, rutas absolutas y relativas, FileReader y BufferedReader

El paquete `java.io` reúne las clases de entrada y salida de Java. Se importa con `import java.io.*;` o clase por clase. Para localizar un archivo se usan rutas:

- **Ruta absoluta:** indica la ubicación completa desde la raíz del sistema, por ejemplo `C:/Users/estudiante/datos/notas.txt`.
- **Ruta relativa:** parte de la carpeta del proyecto, por ejemplo `datos/notas.txt`. Es más portable, porque funciona aunque el proyecto cambie de equipo.

`FileReader` abre un archivo de texto y lo lee carácter por carácter. `BufferedReader` se coloca sobre un `FileReader` y añade un búfer en memoria, lo que hace la lectura más eficiente y habilita `readLine()`.

```
try (BufferedReader br = new BufferedReader(new FileReader("datos/notas.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
} catch (IOException e) {
    System.out.println("Error de lectura: " + e.getMessage());
}
```

### 1.5 Clase File: creación de objetos y métodos exists(), createNewFile(), delete() y getPath()

La clase `File` representa una ruta de archivo o carpeta; crear el objeto no crea el archivo en el disco, solo lo describe. Sus métodos principales son:

- `exists()`: indica si el archivo o la carpeta existe.
- `createNewFile()`: crea el archivo vacío; devuelve `true` si lo creó y puede lanzar `IOException`.
- `delete()`: elimina el archivo; devuelve `true` si lo logró.
- `getPath()`: devuelve la ruta con la que se creó el objeto.

```
File archivo = new File("datos/prueba.txt");
System.out.println("Ruta: " + archivo.getPath());
if (!archivo.exists()) {
    archivo.createNewFile();      // requiere manejar IOException
}
System.out.println("Existe: " + archivo.exists());
archivo.delete();
```

## Tema 2. Escritura y procesamiento de información

### 2.1 write() y close(), sobrescribir o agregar contenido, y manejo de excepciones

El método `write()` envía texto al archivo y `close()` cierra el flujo, garantizando que los datos se guarden realmente en el disco; olvidar cerrarlo puede dejar información sin escribir. El constructor decide qué ocurre con el contenido previo:

- **Sobrescribir** (por defecto): `new FileWriter("datos.txt")` borra lo que había y empieza de cero.
- **Agregar** (modo append): `new FileWriter("datos.txt", true)` conserva el contenido y añade al final.

Como las operaciones de archivo pueden fallar (ruta inexistente, falta de permisos, disco lleno), Java obliga a tratar la `IOException` con `try-catch` o con `throws`. El bloque *try-with-resources* cierra el archivo automáticamente.

```
try (FileWriter fw = new FileWriter("estudiantes.txt", true)) {   // true = agregar
    fw.write("Ana Pérez\n");
} catch (IOException e) {
    System.out.println("No se pudo escribir: " + e.getMessage());
}
```

### 2.2 Escritura de archivos con FileWriter y BufferedWriter

`FileWriter` escribe texto directamente en un archivo. `BufferedWriter` lo envuelve y usa un búfer para reducir los accesos al disco; además ofrece `newLine()`, que inserta el salto de línea correcto para cada sistema operativo.

```
try (BufferedWriter bw = new BufferedWriter(new FileWriter("materias.txt"))) {
    bw.write("Programación");
    bw.newLine();
    bw.write("Cálculo");
    bw.newLine();
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

### 2.3 Entrada, procesamiento y salida de datos al trabajar con archivos

Todo programa con archivos sigue el esquema **Entrada, Proceso, Salida**. La entrada son los datos leídos del archivo o ingresados por teclado; el proceso son los cálculos y decisiones sobre esos datos; la salida es el resultado mostrado en pantalla o guardado en otro archivo. Separar estas tres etapas facilita el diseño del algoritmo, la prueba de escritorio y la depuración.

| Entrada | Proceso | Salida |
|---|---|---|
| Líneas leídas de notas.txt | Convertir, sumar y calcular el promedio | Promedio mostrado y guardado en resultados.txt |

### 2.4 Procesamiento de información, validación de datos y manejo de errores

Procesar información es transformar datos en bruto en resultados útiles mediante cálculos, comparaciones y clasificaciones. Como los archivos pueden contener datos incompletos o incorrectos, es indispensable validarlos antes de usarlos:

- Comprobar que el archivo exista antes de leerlo (`exists()`).
- Verificar que las líneas no estén vacías y que los valores estén dentro de un rango válido (por ejemplo, notas entre 0 y 10).
- Capturar excepciones con `try-catch` y mostrar mensajes claros en lugar de dejar que el programa se detenga de forma abrupta.

### 2.5 Conversión de texto a números, búsqueda y filtrado de información

Todo lo que se lee de un archivo de texto llega como `String`, por lo que para operar matemáticamente hay que convertirlo con `Integer.parseInt()` o `Double.parseDouble()`. Si el texto no es numérico, se lanza una `NumberFormatException` que debe capturarse. Una vez convertidos los datos, se puede **buscar** un registro concreto con `equals()` o `contains()`, y

**filtrar** los que cumplan una condición, por ejemplo las notas aprobadas.

```
String linea = "8.5";
try {
    double nota = Double.parseDouble(linea.trim());
    if (nota >= 7.0) {
        System.out.println("Aprobado");
    }
} catch (NumberFormatException e) {
    System.out.println("Dato no numérico: " + linea);
}
```

### 2.6 Ejercicio integrador: leer notas, calcular el promedio y mostrar resultados

Este ejercicio reúne lo visto en ambos temas: se leen las notas de un archivo, se validan y convierten a número, se calcula el promedio y se muestran los resultados.

```
import java.io.*;
import java.util.Locale;

public class PromedioNotas {
    public static void main(String[] args) {
        double suma = 0;
        int contador = 0;

        try (BufferedReader br = new BufferedReader(new FileReader("notas.txt"))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                try {
                    double nota = Double.parseDouble(linea.trim());
                    if (nota >= 0 && nota <= 10) {
                        suma += nota;
                        contador++;
                    }
                } catch (NumberFormatException e) {
                    System.out.println("Línea ignorada: " + linea);
                }
            }
        } catch (IOException e) {
            System.out.println("Error de lectura: " + e.getMessage());
            return;
        }

        if (contador > 0) {
            System.out.printf(Locale.US, "Notas válidas: %d%n", contador);
            System.out.printf(Locale.US, "Promedio: %.2f%n", suma / contador);
        } else {
            System.out.println("No hay notas válidas.");
        }
    }
}
```

# 3. Conclusiones

- Los archivos permiten que la información de un programa persista más allá de su ejecución, ya que la RAM es volátil y solo el almacenamiento permanente conserva los datos.
- Java organiza la lectura y la escritura en `java.io`: `File` describe y administra el archivo, `FileReader` y `BufferedReader` leen, y `FileWriter` y `BufferedWriter` escriben. Las versiones con búfer son más eficientes y más cómodas de usar.
- Elegir entre sobrescribir y agregar contenido es una decisión de diseño importante, pues un error en el segundo parámetro de `FileWriter` puede borrar información valiosa.
- Las excepciones, la validación de datos y la conversión controlada de texto a números hacen que los programas sean robustos frente a archivos inexistentes, vacíos o con datos incorrectos.
- El esquema entrada, proceso y salida, aplicado en el ejercicio integrador, demuestra que los archivos se integran de forma natural con el resto de la lógica de programación.

# 4. Recomendaciones

- Cerrar siempre los archivos, preferiblemente con *try-with-resources*, para evitar pérdida de datos y fugas de recursos.
- Verificar con `exists()` la existencia del archivo antes de leerlo y manejar la `IOException` con mensajes claros.
- Usar rutas relativas dentro del proyecto para que el programa funcione en cualquier equipo sin modificar el código.
- Revisar el modo de apertura de `FileWriter` (con o sin `true`) antes de ejecutar, o se puede perder el contenido anterior.
- Validar y limpiar (`trim()`) cada dato leído antes de convertirlo con `parseInt()` o `parseDouble()`.
- Practicar con archivos de prueba pequeños y realizar pruebas de escritorio para comprobar el flujo entrada, proceso y salida antes de trabajar con datos reales.